



Instituto Superior Técnico
University of Lisbon

Parallel and Distributed Computing

Document Classification Project Report - MPI

Group 17:

João Miguel Tiago – 99986

Maria João Pupo Correia – 86476

Tomás Simões Cândido – 106470

MSc in Computer Science and Engineering
2nd Semester 2025/2026

April 3, 2026

Contents

1 Parallelization Approach 2

2 Data Decomposition 2

3 Synchronization 2

4 Load Balancing 2

5 Communication Cost 3

6 Performance results 3

6.1 Results 3

6.2 Discussion 3

6.3 Large-scale scalability analysis 4

7 Changes from Previous Delivery 5

1 Parallelization Approach

The algorithm was parallelized using a hybrid MPI+OpenMP model. MPI is used to distribute the dataset across processes, while OpenMP is used to exploit thread-level parallelism within each process. Each MPI rank operates on a disjoint subset of the documents, performing all local computations without requiring point-to-point communication.

The main iterative structure of the algorithm is preserved, with each iteration consisting of three phases: accumulation of document contributions, centroid computation, and reassignment. Local computations are performed in parallel using OpenMP, while global consistency of centroids is ensured through collective communication using `MPI_Allreduce`.

This hybrid approach allows the workload to be distributed across nodes (via MPI) while still leveraging shared-memory parallelism within each node (via OpenMP), which enables scaling beyond a single node while still exploiting shared-memory parallelism within each node.

2 Data Decomposition

Documents are distributed across MPI ranks using a block decomposition. The D documents are divided into `nprocs` contiguous blocks, where ranks 0 to $D \bmod \text{nprocs} - 1$ receive one extra document to handle uneven divisions. Rank r owns documents in the range $[\text{rank_start}[r], \text{rank_start}[r] + \text{rank_numdocs}[r][$, where $\text{rank_start}[r] = r \times (D/\text{nprocs}) + \min(r, D \bmod \text{nprocs})$. Each rank allocates only its local slice of the document array, so the full dataset is never held on a single node, a requirement given that the dataset may not fit in a single node's memory.

Each rank generates its local documents independently using a shared pseudo-random seed (`SEED = 1234`). To ensure each rank produces the correct slice of the global sequence, each rank advances the random number generator by $\text{rank_start}[r] \times S$ steps before sampling its own documents. This avoids any file data beyond reading the header, and ensures deterministic and consistent data generation across all ranks without communication.

The centroid table ($C \times S$) is small and replicated on every rank, as it must be identical across all processes for reassignment to be consistent.

3 Synchronization

Global centroid consistency is achieved through two `MPI_Allreduce` calls with `MPI_IN_PLACE` reduce `local_sums` ($C \times S$ doubles, `MPI_SUM`) and `local_cnts` (C doubles, `MPI_SUM`) globally, so every rank ends up with the same centroid sums and counts in-place, without needing separate send and receive buffers. The centroids and their norms are then computed locally within the same single block. The implicit barrier at the end of `omp single` ensures all threads see the updated centroids before the reassignment loop begins.

Convergence is detected by accumulating `thread_reassigned` per thread into the shared `total_reassigned` counter via `#pragma omp atomic`, then reducing across all ranks with a final `MPI_Allreduce` (`MPI_SUM`). The loop continues as long as `total_reassigned > 0` globally.

4 Load Balancing

Load balancing is addressed through the block distribution of documents across MPI ranks, ensuring that each process receives a similar number of documents. Within each process, OpenMP uses static scheduling to evenly distribute iterations among threads. Since the workload per document is uniform, this results in good load balance with minimal scheduling overhead.

5 Communication Cost

Each iteration requires three `MPI_Allreduce` calls with message sizes $C \times S$ doubles, C doubles, and 1 integer respectively. The use of `MPI_IN_PLACE` avoids allocating separate output buffers and reduces memory pressure, at the cost of the input buffer being overwritten in-place. The dominant cost is the first reduction over $C \times S$ doubles, which grows with the number of cabinets and features but is independent of D . This means communication volume does not increase as more ranks are added (and D per rank decreases), which is a favourable property for scalability. As `nprocs` increases, each rank performs less local computation per iteration while the communication cost remains constant, this becomes the primary factor limiting strong scaling at high process counts, as the computation per rank decreases while the cost of collective communication remains unchanged.

6 Performance results

All experiments were conducted on the Deucalion HPC cluster, using the `normal-x86` partition (2 $\times$ AMD EPYC 7742, 64 cores/socket, 2 threads/core, 2 NUMA nodes). Three baseline instances were evaluated: `ex1000-50000-200`, `ex10-200000-100` and `ex200-400000-10`.

The hybrid MPI implementation was executed with 1, 4, 16, and 64 MPI ranks, each using 6 OpenMP threads (`--cpus-per-task=6`), resulting in 6, 24, 96, and 384 total workers. The serial implementation was used as baseline, with speedup defined as $S_p = T_{\text{serial}}/T_{\text{parallel}}$, and efficiency as $S_p/(r \cdot t)$.

6.1 Results

Table 1 presents execution times, speedups, and efficiencies. The corresponding speedup and efficiency trends are illustrated in Fig. 1.

Implementation (rank $\times$ threads)	Target S_p (linear)	ex1000-50000-200			ex200-400000-10			ex10-200000-100		
		time (s)	speedup	efficiency	time (s)	speedup	efficiency	time (s)	speedup	efficiency
Serial	1	189.4	1.00	-	114.0	1.00	-	234.4	1.00	-
1 rank $\times$ 6 threads	6	28.6	6.62	1.10	23.5	4.85	0.81	3.9	60.10	10.02
4 ranks $\times$ 6 threads	24	7.2	26.31	1.10	5.9	19.32	0.81	1.0	234.40	9.77
16 ranks $\times$ 6 threads	96	1.9	99.68	1.04	1.8	63.33	0.66	0.3	781.33	8.14
64 ranks $\times$ 6 threads	384	0.8	236.75	0.62	0.8	142.50	0.37	0.4	586.00	1.53

Table 1: Execution time, linear target (rank $\times$ threads), measured speedup, and parallel efficiency for the three problem instances.

6.2 Discussion

The MPI implementation shows consistent strong scaling up to $P = 96$, after which efficiency degrades due to communication and synchronization overheads (Fig. 1). For `ex1000-50000-200`, near-linear speedup is observed up to 96 workers, partly due to improved cache locality from partitioning the dataset. At $P = 384$, efficiency drops (0.62) as the per-rank workload becomes small (~ 130 documents), making the cost of `MPI_Allreduce` over $C \cdot S = 200\,000$ elements and OpenMP synchronization dominant.

For `ex200-400000-10`, scaling is weaker. The computational intensity is low ($S = 10$), so the cost of thread-level reductions (critical section) and barriers becomes significant relative to useful work, leading to reduced efficiency (0.37 at $P = 384$).

The instance `ex10-200000-100` shows non-comparable speedups due to different data generation between serial and MPI versions, affecting convergence and total work. Additionally, the small com-

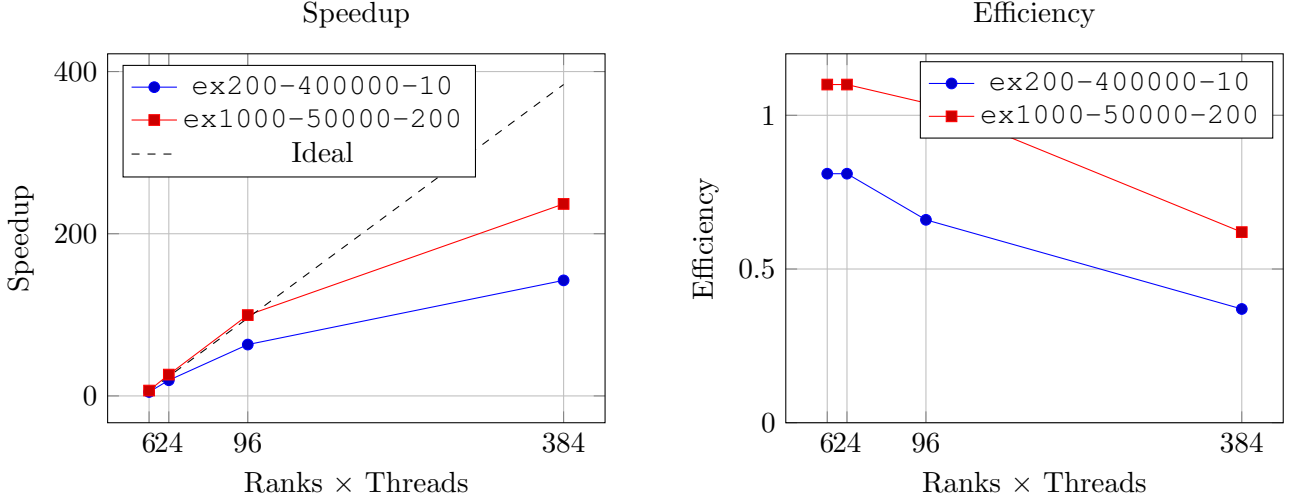


Figure 1: Speedup (left) and efficiency (right) as a function of total parallelism (ranks $\times$ threads). The instance ex10-200000-100 is excluded due to non-comparable behavior.

munication volume ($C \cdot S = 1000$) and large document set amplify cache locality effects, leading to apparent superlinear behavior at moderate P , followed by degradation at high parallelism.

Overall, scalability is governed by the ratio between computation and communication. The computational cost decreases proportionally with P , while the communication cost remains constant per iteration. As a result, maintaining efficiency requires increasing the problem size with the number of processes. The experimental results are consistent with this behavior: instances with higher computational intensity (ex1000-50000-200) scale more efficiently, while instances with lower per-rank workload saturate earlier.

6.3 Large-scale scalability analysis

To evaluate scalability, a larger instance ($C = 10000$, $D = 10^6$, $S = 100$) was executed with multiple rank-thread configurations. Unlike previous sections, the baseline is not a purely serial execution, but a configuration of 1 rank $\times$ 12 threads (12 workers). This choice isolates scalability effects while avoiding the prohibitively long runtime of a fully serial version. Speedup is therefore defined relative to this baseline.

Results show that scalability is primarily driven by increasing the number of MPI ranks. For a fixed total parallelism, configurations with more ranks consistently outperform those with more threads. This is evident when comparing 6×64 (273.1 s) with 64×6 (308.6 s): despite equal total workers, the rank-dominated configuration is $\sim 13\%$ faster. This behavior reflects improved data distribution and reduced memory contention, as each rank operates on a smaller local dataset.

At low parallelism, 12×1 outperforms 1×12 , indicating that OpenMP avoids communication overhead when the workload per rank is large. However, as parallelism increases, MPI scaling dominates, enabling near-linear speedup. Deviations from linear scaling at higher configurations (e.g., 12×256) are explained by the fixed cost of collective communication (MPI Allreduce) and synchronization, which become significant as the per-rank workload decreases.

The results highlight that scalability is achieved primarily through increasing the number of MPI ranks rather than threads. This aligns with the underlying architecture (2 NUMA nodes, 128 physical cores): distributing work across processes improves memory locality and reduces contention, while excessive threading leads to diminishing returns due to memory subsystem limitations.

Threads	Ranks	Workers	Time (s)	S_p	Linear S_p
12	1	12	6777.9	1.00	1
1	12	12	7207.6	0.94	1
6	4	24	3390.9	2.00	2
12	4	48	1798.7	3.77	4
6	16	96	1025.9	6.61	8
128	1	128	896.8	7.56	10.7
12	16	192	461.4	14.69	16
64	4	256	446.1	15.19	21.3
64	6	384	308.6	21.96	32
6	64	384	273.1	24.82	32
12	64	768	165.0	41.08	64
12	128	1536	119.9	56.53	128
12	256	3072	89.5	75.73	256
12	264	3168	87.9	77.1	264
6	1024	6144	60.9	111.30	512

Table 2: Scalability results for the large instance, using 12 threads $\times$ 1 rank as baseline.

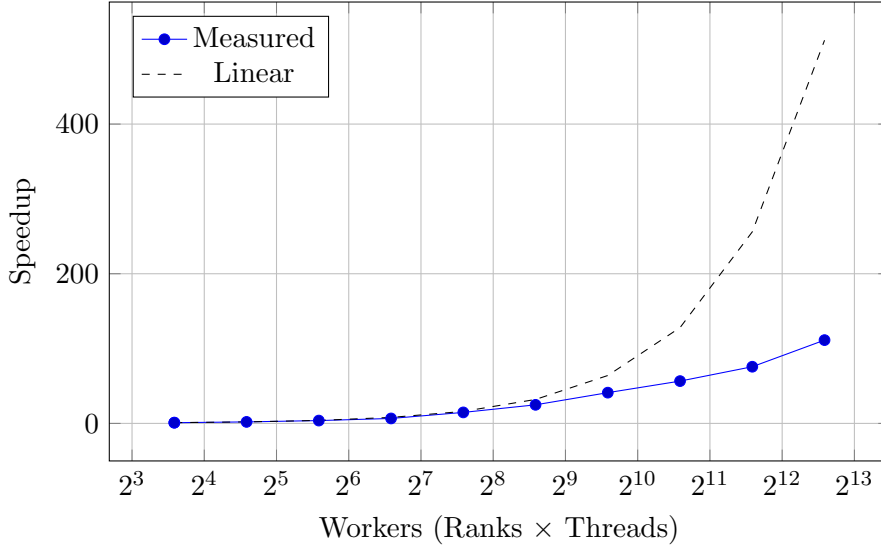


Figure 2: Measured vs linear speedup for the large instance.

7 Changes from Previous Delivery

The implementation was restructured both at the serial and OpenMP levels to eliminate irregular computation patterns and reduce per-iteration overhead.

In the serial version, the previous selective recomputation strategy based on `cabinetsChanged` was removed. That approach relied on a $C \times (D + 1)$ structure storing change flags and cached distances, introducing heavy execution and non-contiguous memory accesses. Its performance was highly data-dependent: when few documents changed assignment, large portions of the computation were skipped, reducing work per iteration; however, when many cabinets changed, the algorithm effectively recomputed most distances while still paying the overhead of maintaining the auxiliary structure and conditional checks. This resulted in unstable performance, with cases significantly

faster or slower depending on the input, making it an inconsistent and unreliable baseline for parallel speedup evaluation.

The updated version performs full recomputation at each iteration. Memory usage is reduced and all conditional checks in the main loops are eliminated.

Distance computation was also reformulated. Instead of explicitly computing:

$$\delta = \sum_{i=0}^{S-1} (d_i - c_i)^2,$$

the expression is expanded as:

$$\delta = \sum d_i^2 + \sum c_i^2 - 2 \sum d_i c_i.$$

Since $\sum d_i^2$ is constant for a given document, it is omitted. The implemented metric becomes:

$$\tilde{\delta} = \sum c_i^2 - 2 \sum d_i c_i,$$

which preserves ordering while reducing computation. Centroid norms are precomputed once per iteration, and the inner loop is reduced to a dot product.

The OpenMP version was aligned with these changes. Thread-local accumulation was preserved, but initialization was optimized by replacing full-array zeroing loops with `memset` on contiguous per-thread slices, improving cache efficiency. Additionally, `calloc` was replaced by `malloc` with explicit initialization, avoiding unnecessary zero-initialization at allocation time.